UNIVERSIDAD ANDRÉS BELLO
FACULTAD DE INGENIERÍA

ENTREGA 2

Backend seguro

Chocomanía: Sistema de Ventas Online

Caso 19 adaptado a una chocolatería.
API Gateway, servicios internos, GraphQL
y pruebas de seguridad y carga.

ASIGNATURA
Desarrollo Web y Móvil
Sección 8750

DOCENTE
Ernesto Vivanco

INTEGRANTES
Florencia Pacheco
Karla Muñoz

FECHA
9 de noviembre de 2026

Informe Entrega 2: Backend seguro

Índice

1. Resumen
2. M1. Matriz de servicios y contratos
3. M2. Modelo del dominio
4. M3. Matriz de sistemas y componentes
5. M4. Stakeholders y matriz de permisos
6. M5. Gateway y fronteras de confianza
7. M6. Matriz de rutas e inventario de secretos
8. M7. Datos sensibles y pruebas de seguridad
9. M8. GraphQL Gateway
10. M9. Caracterización de carga y decisiones de arquitectura
11. M10. Backend completo e integración verificable
12. Matriz de requisitos
13. Brechas declaradas
14. Anexos y evidencias

Resumen

Chocomanía es la tienda online de una chocolatería pequeña. En esta entrega se construyó el backend detrás del front de la Entrega 1: un API Gateway como única puerta de entrada, cinco servicios internos en una red privada (auth, inventory, orders, payments y graphql-gateway), Vault para los secretos del gateway y MongoDB como base. El cliente puede comprar de punta a punta desde el navegador: ver el catálogo, iniciar sesión, crear el pedido, pagarlo, ver la boleta y revisar sus pedidos por GraphQL.

Lo que cruza todo el sistema:

- **Una sola puerta:** solo el gateway publica un puerto (8000) y solo deja pasar las rutas de su lista blanca.
- **Correlación:** cada solicitud lleva un `X-Request-ID` que aparece en la respuesta, en los logs JSON de cada servicio y en lo que se guarda en la base.
- **El servidor decide:** precios, totales, montos, estados, cajero y boleta los calcula el backend; lo que mande el navegador se ignora.
- **Un secreto por integración** y propiedad de los recursos revisada en cada servicio, no solo en el gateway.
- **Idempotencia** en pedidos y pagos (`Idempotency-Key`).

M1. Matriz de servicios y contratos

Documento: `m1-servicios.md`

13 requisitos (R01 a R13) tomados del Caso 19, 9 flujos (F01 a F09) y 19 servicios (S01 a S19). Cada servicio tiene su requisito, su flujo, la pantalla que lo usa, el acceso, la clasificación (consulta, transaccional, integración, agregación o analítica), el estado de implementación y un ejemplo JSON real tomado del seed o de las evidencias. Las reglas comunes cubren el formato único de error, la validación (422), la correlación, la identidad, el recurso ajeno (404), la idempotencia y qué pasa si un servicio cae.

17 de los 19 servicios están implementados. S09 y S10 (servicio clients) quedan como brecha.

M2. Modelo del dominio

Documentos: `m2-dominio.md` y `uml-dominio.drawio`

Diagrama de clases con Usuario, Rol, Cliente, Producto, Pedido, LineaPedido, Boleta y Pago, sus multiplicidades (por ejemplo, Pedido y LineaPedido 1 a 1..* por composición, Pedido y Pago 1 a 0..*), 13 reglas del dominio (D01 a D13) y la máquina de estados del pedido. Confirmado con el código de orders y payments. Cliente queda marcado como no implementado en la U2.

M3. Matriz de sistemas y componentes

Documentos: `m3-sistemas.md` y `componentes.drawio`

Cada sistema con su responsabilidad, sus datos y su tipo de dependencia (real, doble de prueba o no implementado). Despacho y boletas viven dentro de orders y el proveedor de pago es un doble de prueba que respeta el contrato. Incluye las reglas entre sistemas, la secuencia completa de una compra (crear, pagar y ver mis pedidos) ligada a sus evidencias, y los límites conocidos (por ejemplo, la devolución de stock sin reintento).

M4. Stakeholders y matriz de permisos

Documentos: `m4-stakeholders.md` y `m4-permisos.md`

Registro de 11 stakeholders con su interés e influencia. Matriz de permisos por servicio y rol (Anónimo, CLIENTE, ADMIN, DUENO, CAJERO, DESPACHO), las operaciones internas que no expone el gateway (reservas de stock, marcar pagado, lectura para GraphQL) con el secreto que exige cada una, y las reglas que salen de la matriz.

M5. Gateway y fronteras de confianza

Documentos: `m5-gateway.md` y `uml-gateway.drawio`

Por qué un solo gateway, el UML de sus clases (incluidas GraphQLGateway, Schema y Resolver), la clasificación de rutas (anónimas, autenticadas, por rol, integración y front estático) y las fronteras de confianza entre navegador, gateway, servicios internos, graphql-gateway y proveedor. Muestra el recorrido de una solicitud autorizada (S11, `crear-pedido-0001`, en cuatro servicios) y el de una denegada (S03 como CLIENTE).

M6. Matriz de rutas e inventario de secretos

Documento: `m6-rutas-secretos.md`

Matriz de todas las rutas públicas con método, acceso, servicio destino y controles; inventario de los secretos (uno por integración, quién lo tiene y dónde vive); qué hacer si se filtra una clave; qué pasa si Vault no responde (503 `SECRETS_UNAVAILABLE`, caché de 60 s); y la revisión de secretos en el repo, capturas y logs.

M7. Datos sensibles y pruebas de seguridad

Documento: `m7-datos-sensibles.md`

Cinco niveles de clasificación y una matriz de datos con finalidad, custodio, roles, protección, retención y estado real en la U2. Chocomanía no custodia secretos del cliente (no recibe datos de tarjeta). Incluye la minimización, el recorrido de la dirección de despacho, la verificación de correo simulada, las 6 pruebas de seguridad (P01 a P06, todas aprobadas con pytest en E07) y los riesgos de datos que quedan abiertos.

M8. GraphQL Gateway

Documento: `m8-graphql.md`

GraphQL Gateway privado (Strawberry) detrás del API Gateway, con las queries `misPedidos` y `pedido`. Junta orders e inventory con DataLoader (una sola llamada a inventory por consulta), limita profundidad (6), alias (10), tokens (1000) y paginación (20), autoriza por sesión, rol, recurso y campo (`stock` solo ADMIN), y devuelve errores parciales: si inventory cae, los pedidos igual se ven y `producto` viene null con `UPSTREAM_UNAVAILABLE`. Lo usa `mis-pedidos.html`.

M9. Caracterización de carga y decisiones de arquitectura

Documentos: `m9-carga.md`, `adr-grpc.md` y `adr-analitica.md`

Carga proyectada de un local pequeño: hasta 500 pedidos y 10.000 consultas al catálogo por día en peak, menos de 25 solicitudes por segundo. Umbrales fijados antes de medir (p95 del catálogo menor a 500 ms, de crear pedido menor a 1.000 ms, errores menores a 1 %). Script k6 del catálogo en `tests/carga/catalogo.js`.

Medición (E08): la primera ejecución no cumplió los umbrales (p95 de 3,2 s y 22 solicitudes por segundo) porque el gateway creaba un cliente HTTP por solicitud (INC06). Con el arreglo, el catálogo cumple con holgura: p95 de 19 ms, 65 solicitudes por segundo y 0 % de errores. Los umbrales no se cambiaron.

- **ADR gRPC:** no se usa. Con esta carga, HTTP con JSON entre servicios alcanza y es más fácil de depurar y correlacionar.
- **ADR analítica:** el reporte de ventas es un endpoint dentro de orders (S19), no un servicio aparte.

M10. Backend completo e integración verificable

Qué está integrado

Componente	Qué hace	Evidencia
gateway	Lista blanca, sesión por cookie, identidad hacia los servicios, `X-Request-ID`, logs JSON, errores con formato único, secretos desde Vault, front estático en el mismo origen, un solo cliente HTTP (INC06)	E01, E07, E08
auth	Usuarios y sesiones en Mongo, bcrypt costo 12, token guardado como hash, TTL de 900 s	E02 a E06 (`/introspect` en cada traza)
inventory	Catálogo, stock, reservas y devoluciones internas, lote para GraphQL	E01, E02, E05
orders	Pedidos con idempotencia, precio del servidor, cajero virtual, anulación con motivo, cola de despacho, estados, boleta y reporte de ventas	E02, E03
payments	Pago con idempotencia, proveedor simulado (inmediato o aviso), reconciliación si orders cae	E03, E04
graphql-gateway	`misPedidos` y `pedido` con DataLoader, límites y errores parciales	E05, E06
front	Catálogo, carrito, login, checkout, pago, boleta y mis pedidos contra el backend real	E09 (escritorio, celular y error parcial)

Cómo verificarlo

1. Copiar `.env.example` a `.env` y completar los valores (en Windows, `scripts/generar-env.py` lo crea con secretos aleatorios).
2. `docker compose up -d --build` y esperar a que los servicios queden `healthy`.
3. `docker compose run --rm seed --reset` (o `scripts/reset.sh`) para cargar el seed (6 usuarios y 5 productos con ID fijos).
4. Abrir `http://localhost:8000`, iniciar sesión con `cliente@correo.cl` y comprar un producto.

5. Copiar el `X-Request-ID` de cualquier respuesta y ejecutar `scripts/logs.sh <request_id>` para ver la traza en todos los servicios.
6. Revisar la base con `docker compose run --rm check pedido PED-0003`.
7. Correr las pruebas automáticas con `docker compose run --rm pruebas` (16 pruebas, E07) y la carga con `docker compose run --rm k6` (E08), cada una sobre la base reseteada.

Lo que no está en la U2

El servicio clients (registro y perfil), el panel interno (mantenedor, cola de despacho y reporte no tienen pantalla, aunque sus servicios funcionan), el envío de la boleta por correo, el límite de solicitudes y las pruebas con Playwright. Ver Brechas declaradas.

Historial

Cada cambio está en `bitacora.md` (INCR-00 en adelante) y las fallas encontradas en `incidencias.md`.

Matriz de requisitos

Documento: `matriz-requisitos.md`

Une cada requisito con su pantalla, servicio, prueba, resultado y evidencia. R01 a R04, R07 y R12 tienen evidencia aprobada, y R08 en parte; R09, R10 y R13 tienen el servicio implementado pero sin evidencia (no tienen pantalla); R05, R06 y R11 son brechas.

Brechas declaradas

Brecha	Motivo	Módulo donde se declara
Servicio clients no implementado (S09, S10): el registro y el perfil siguen en localStorage, con la clave en texto plano (INC05)	Recorte por tiempo; se priorizó la compra completa	M1, M7
R06 (registro por el administrador) y R10 para usuarios y clientes	Dependen de clients	M1
Verificación del correo simulada, sin API de mensajería (R11)	Recorte acordado	M7
Boleta sin envío por correo (R08 parcial)	Sin servicio de mensajería	M1, M2
Panel interno sin pantallas: mantenedor de productos, cola de despacho y reporte de ventas solo por API	Recorte por tiempo	M1, M10
Proveedor de pago simulado (doble de prueba, no Webpay)	Recorte acordado	M3
Solo el gateway lee secretos de Vault; Vault en modo dev	Entorno local	M6
Todos los servicios usan el usuario raíz de Mongo	Entorno local	M6, M7
Cookie sin `secure` en local	No hay HTTPS en local	M6, M7

Brecha	Motivo	Módulo donde se declara
Sin límite de solicitudes (rate limit)	Pasa a la Etapa 4	M6
Sin pruebas con Playwright	Pasa a la U3; en la U2, evidencia manual	Protocolo de pruebas
Stock descontado sin pedido si inventory responde después del plazo de orders	Requiere reservas con ID propio	M3
GraphQL: `stock` no lo puede ver nadie (las queries son solo de CLIENTE), lote máximo de 50 productos e introspección activa	Decisiones de alcance	M8
Cuentas de prueba con sus claves visibles en el login	Facilitan la demo; son ficticias	M7
CPU y memoria no medidas en la prueba de carga; orders, payments y graphql-gateway todavía crean un cliente HTTP por solicitud	Solo se sometió a carga el catálogo	M9
Sin evidencia de la cola de despacho, el reporte de ventas ni el mantenedor de productos como ADMIN	No tienen pantalla en la U2	Matriz de requisitos

Anexos y evidencias

- Índice de evidencias: `../evidencias/indice.md` (E01 a E09, cada una con su ficha).
- Protocolo de pruebas: `protocolo-pruebas.md`.
- Incidencias: `incidencias.md`.
- Bitácora de incrementos: `bitacora.md`.
- Reparto de roles: `reparto-roles-entrega-2-v2.md`.